



2

6

720 sp

عملي

كلية الهندسة المعلوماتية

السنة الأولى

البنى الشرطية 1

د. ماهر الصارم



RB Informatics; 04/11/2024

برمجة C++

أهلاً بكم في محاضرة الثانية برمجة TD- تعلمنا في المحاضرة السابقة كيفية تحويل خوارزمية إلى لغة برمجة، وفي هذه المحاضرة سنتعلم استخدامات تعليمية البنى الشرطية (كيف نضع شرط) .

- بداية يجب أن نتذكر أنه لكتابة أي مسألة برمجية سنمر بالمراحل الآتية:
- 1. نحدد مدخلات ومخرجات
- 2. كتابة خوارزمية الحل
- 3. ثم نحول إلى كود برمجي.

لدينا معادلة رياضية $A = (2B + 2CD) / 2C$ اكتب خوارزمية ومن ثم برنامج لحالها

الخوارزمية:

ادخل B, C, D

احسب $A = (2B + 2CD) / 2C$

اطبع A

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int B, C, D;
5     int A;
6     cout << "Enter B, C, D and C≠0" << endl;
7     cin >> B >> C >> D;
8     A = (float) (2 * B + 2 * C * D) / (2 * C);
9     cout << "A=" << A << endl;
10    return 0; }
```

للتذكير: قسمة عدد صحيح على عدد

صحيح هو عدد صحيح . في حال أردنا

الخرج أن يكون عدد عشري لدينا

طريقتين :

نضع (float) كما في مثال

أو $A = (float) (2 * B + 2 * C * D) / (2 * C);$

نصرب ب : (1.0)

$A = 1.0 * (2 * B + 2 * C * D) / (2 * C);$

شرح:

الأسطر من 1 <= 3 هي أسطر أساسية بلغة ++c .

السطر 4: عرفنا المتحولات المدخلة من نوع int

السطر 5: عرفنا المخرجات من نوع (float) لأن ناتج القسمة يمكن أن يكون عشري

السطر 6: طبعنا عبارة توضح للمستخدم أنه يجب أن يدخل B,C,D و $c \neq 0$ (لا يمكن C ان يكون صفر لأن في هذه الحالة سيعطي error غير معرف)

السطر 7: ادخال D,C,B

السطر 8: حساب A ووضعه (float) قبل العملية الحسابية لنخزن قيمة عشرية داخل المتحول A في حال عدم وضع (float) سيتم تخزين قيمة int او صحيحة في المتحول A

السطر 9: طباعة قيمة A.

البنى الشرطية : تعني معالجة الحل البرمجي الذي يحوي على شرط , فيسلك البرنامج إحدى مسارين إما الشرط محقق فتتخذ التعليمات الخاصة بالشرط أو الشرط غير محقق فلا تتخذ التعليمات الخاصة بالشرط .

نعتبر عن البنى الشرطية برمجياً بلغة ++c بطريقتين :

1- if - else 2- switch - case

سنتناول في هذه المحاضرة أول نوع وهو ال if - else .

أشكال تعليمة ال if - else :

الشكل الأول:
(شرط) if
{ مجموعة تعليمات }
==>



حيث نتيجة الشرط هي نتيجة بوليانية أي :

إما true ← ينفذ التعليمات الخاصة بالشرط .

أو false ← لا ينفذ التعليمات الخاصة بالشرط .

أمثلة تدريجية عن بنية if الشرطية :

تمرين 1: أكتب برنامج لاختبار فيما إذا كان العدد الصحيح x المدخل سالب أم موجب:

الخوارزمية :

- أدخل x من قبل المستخدم .
- إذا كان $(x \geq 0)$
- اطبع " x موجب "
- اطبع " Done "
- انتهى برنامج

عند تنفيذ البرنامج:

```

1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.     int x;
6.     cout << "enter x" << endl;
7.     cin >> x;
8.     if(x>=0){
9.         cout << "Hello" << endl;
10.        cout << "x is positive" << endl;
11.        cout << "Done " << endl;}
12. return 0;}

```

إذا أدخل المستخدم قيمة أكبر أو تساوي 0 ولنفرض أنه أدخل $x=10$ سيختبر الشرط الأول (هل x أكبر أو تساوي 0؟) (نعم) فالشرط محقق و سوف يطبع على الشاشة :

إضاءة : لو وضعنا (;) بعد if فإن compiler سينفذ من جميع التعليمات في حال كان الشرط محقق أم لا مثلاً:
لو أدخلت $x=-1000$ سيكون الخرج :

```

Hello
x is positive
Done

```

ملاحظات هامة:

- إذا تحقق الشرط سيتم تنفيذ ما بين أقواس المجموعة {...}.
- إذا لم نضع أقواس لن يعطي error ولكن سينفذ فقط أول تعليمة بعد الشرط.
- لا نضع (;) بعد الشرط , لأن كل فاصلة منقوطة تعني نهاية التعليمة , فإذا كان بعد تعليمة الشرط (if) فاصلة منقوطة فإنه سينفذ التعليمات التي تليه في حال تحقق الشرط أو لم يتحقق.
- نستخدم {} ب (if - else - while - for) و نسميها ال Block .
- أقواس المجموعة {...} اختيارية في حال كان هناك تعليمة واحدة فقط أما في حال كان هناك عدة تعليمات فيجب وضعها ليتم تنفيذ جميع التعليمات.

الشكل الثاني:

<==

```

(شرط) if
{ (مجموعة تعليمات) }
else
{ (مجموعة تعليمات) }

```

ملاحظات هامة:

- else ليست اجبارية كما لاحظنا أنه يمكن استخدام if بدونها.
- بعد else لا يوجد شرط.
- لا يمكن استخدام else ما لم يوجد if بعدها.



■ تمرين 2 : اكتب خوارزمية وبرنامج يختبر فيما إذا كان العدد زوجي أم فردي :

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.     int x;
6.     cout << "enter x: " << endl;
7.     cin >> x;
8.     if(x%2==0)
9.         cout << x << "is even" << endl;
10.    else
11.        cout << x << "is odd" << endl;
12.    return 0;}
```

خوارزمية:

أدخل x .

اختبر $x \% 2$ (باقي قسمة x على العدد 2)

فإذا كان باقي القسمة يساوي ال صفر ← اطبع x عدد زوجي .

و إلا ← اطبع x عدد فردي .

ملاحظة :

== ← مقارنة

= ← إسناد

- إذا أردنا أن نختبر الشرط $if(x==1)$ وبدلاً من أن نكتب $if(x=1)$ كتبنا $if(x=1)$ ماذا سيحدث؟
هنا نكون أسندنا القيمة 1 إلى x فالشرط دائماً محقق وبالتالي ستنفذ التعليمة الخاصة بالشرط دائماً .
- من الممكن أن نضع if داخل if :

```
if (شرط) {
    if (شرط)
        {تعليقات}
}
```
- إذا كان الشرط مقسم لمجالات , يكون استخدام $if-else$ أفضل من استخدام عدة شروط (if) مستقلة لأن تنفيذ البرنامج سيكون أسرع .

■ تمرين 3: اكتب برنامج لحل معادلة من الدرجة الأولى ($ax+b=0$) :

```
1#include <iostream>
2using namespace std;
3int main() {
4    int a,b;
5    float x;
6    cin>>a>>b;
7    if(a==0) {
8        cout<<"impeessbile"<<endl;
9    } else {
10        x=-(float)b/a;
11        cout<<"x="<<x;
12    }
13    return 0; }
```




```

    if (شرط)
    {
        (مجموعة تعليمات)
    }
    else if (شرط)
    {
        (مجموعة تعليمات)
    }
    else
    {
        (مجموعة تعليمات)
    }

```

الشكل الثالث:

<==

■ تمرين 1 : أكتب برنامج لاختبار فيما إذا كان العدد الصحيح x المدخل سالب أم موجب

الخوارزمية:

```

1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.     int x;
6.     cout << "enter x" << endl;
7.     cin >> x;
8.     if(x>=0)
9.         cout << "x is positive" << endl;
10.    else if(x<0)
11.        cout << "x is negative" << endl;
12.    else {
13.        cout << " Zero " << endl;
14.    }
15.    return 0;

```

- أدخل x من قبل المستخدم .
- إذا كان $(x \geq 0)$
- اطبع " x موجب "
- إذا كان $(x < 0)$
- اطبع " x سالب "

عند تنفيذ البرنامج:

إذا أدخل المستخدم قيمة أكبر أو تساوي 0 ولنفرض أنه أدخل $x=10$ سيختبر الشرط الأول (هل x أكبر أو تساوي 0 ؟) (نعم) فالشرط محقق و سوف يطبع على الشاشة x is positive ثم سينزل سطر ، و سيختبر الشرط الثاني (هل x أصغر من 0 ؟) (لا) فلا تنفذ التعليمة الخاصة بالشرط الثاني. وبالعكس إذا أدخل قيمة سالبة .

■ تمرين 2 : أكتب برنامج لاختبار فيما إذا كان العدد سالب أم موجب (نفس المثال السابق) لكن باستخدام if-else :

ما الفرق بين هذا المثال والمثال السابق؟

الفرق هو أن هذا البرنامج أسرع من ذلك ، لأن البرنامج الأول سيختبر الشرط الأول و يختبر الشرط الثاني (سيختبر كل شرط على حدة) . أما في هذا البرنامج سيختبر تعليمة if إذا تحققت سينفذها ويقوم بتجاهل التعليمات التي تتبعها ، أي سيتخطى else و لن يختبرها و بالتالي وقت أقصر .

ملاحظة :

في تعليمة if إذا لم يتحقق الشرط فسيخرج من البنية الشرطية كلها . أما if-else إذا لم يتحقق الشرط سينفذ تعليمات else .

```

1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.     int x;
6.     cout << "enter x" << endl;
7.     cin >> x;
8.     if(x>=0)
9.         cout << "x is positive" << endl;
10.    else
11.        cout << "x is negative" << endl;
12.    return 0;}

```



سنختتم محاضرتنا ببعض فوائد البرمجة

- `int ,return 0;` ليس مهم كتابتها في برنامج لان هي أساسا موجودة (by defulte) .
- أنواع أخطاء البرمجة:
- خطأ الترجمة compiler error : يتعلق بالعبارات التي يكتبها المبرمج مثل : نسيان (;) كتابة كلمات مفتاحيه بشكل خطأ نسيان استدعاء مكتبة.
- خطأ التنفيذ: (Run Time Error) لا يوجد مشكلة بالبرامج وإنما بالتنفيذ مثل : القسمة على صفر في مصفوفات index يبدأ من (0) وإذا وضعنا index سالب يعطي error .
- خطأ المبرمج : يعطي خطأ عندما ينفذ مثل: أخطاء في كتابة العمليات الحسابية.

إشارات المقارنة في البرمجة :

> أكبر

< أصغر

>= أكبر أو يساوي

<= أصغر أو يساوي

= يساوي

!= لا يساوي || فائدة اربيسيزية || هناك طريقة أخرى لكتابتك `if (condition)` وهي تدعى (short if)

`syntax =(conditions True? True:false);` كالشكل الآتي:

`cout<<(x>=0 ? "positive": "negative")` , ويمكني أيضا اسناد هذا شرط لمتحول وطباع قيمة متحول:

`string s= (x>=0) ? "positive": "negative";`

`cout<<s;`